



1. Généralité

A. Pourquoi la POO

La programmation orientée objet (POO) présente plusieurs avantages par rapport à d'autres paradigmes de programmation comme la programmation procédurale ou fonctionnelle. Voici les principaux :

- 1) **Modélisation plus proche du monde réel** : la POO permet de représenter les entités du monde réel sous forme d'objets avec des **attributs** (états) et des **méthodes** (comportements).

Cela rend le code plus intuitif, surtout pour des systèmes complexes.

Exemple

Une classe **Voiture** avec des attributs comme **couleur**, **vitesse**, et des méthodes comme **démarrer()**, **freiner()**.

- 2) Réutilisation du code grâce à l'héritage : l'**héritage** permet de créer de nouvelles classes à partir de classes existantes. On évite la duplication de code, on le rend plus DRY (Don't Repeat Yourself).

Exemple

Une classe **Vehicule** peut être utilisée comme base pour créer **Voiture**, **Moto**, **Camion**.

- 3) Encapsulation : l'**encapsulation** permet de cacher l'implémentation interne d'un objet et de ne fournir qu'une interface d'utilisation. Cela protège les données et réduit les risques d'erreurs.

Exemple

On peut interdire l'accès direct à un attribut sensible et ne le modifier qu'à travers des méthodes.

- 4) Modularité : le code est organisé en objets indépendants, ce qui rend le développement, le test, la maintenance et la compréhension plus faciles.

Exemple

Il est possible de modifier une classe sans impacter les autres, si les interfaces restent inchangées.

- 5) Polymorphisme : cela permet d'utiliser le même nom de méthode pour des objets différents, qui réagiront chacun à leur manière. Cela rend le code plus souple et évolutif.

Exemple

Une méthode **afficher()** peut exister dans plusieurs classes (ex. : **Cercle**, **Carré**), chacune ayant sa propre implémentation.

- 6) Facilité de maintenance et d'évolution : le découpage en objets autonomes facilite la maintenance et la montée en complexité des projets.

Le code est plus lisible, structuré, évolutif.

- 7) Les équipes peuvent travailler sur différentes classes ou modules en parallèle.

- 8) Favorise la réutilisation et les bibliothèques génériques et réutilisables.

Elle est souvent utilisée dans les frameworks modernes (ex. : Django, Spring, Unity...).

■ DÉFINITION

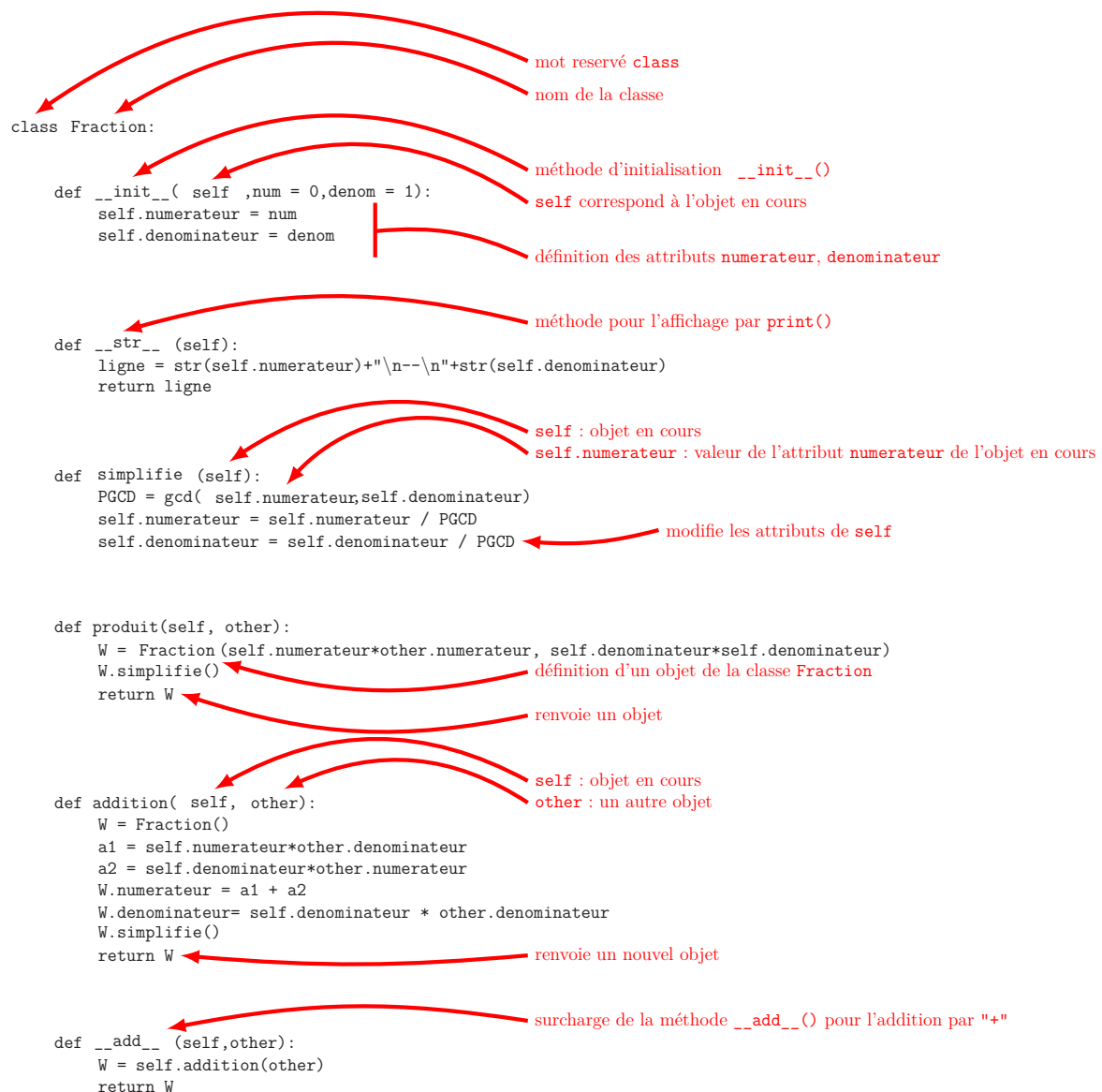
Un objet est une structure de données abstraite regroupant :

- des données associées à l'objet que l'on appelle des **attributs**.
- des fonctions (ou procédures) s'appliquant sur l'objet que l'on appelle **méthodes**.

Ces attributs et méthodes sont réunis dans ce qu'on appelle une **classe** qui est donc un modèle décrivant un objet. Les objets sont ensuite créés à partir de cette classe (ce modèle) qui est donc aussi une machine à fabriquer des objets.

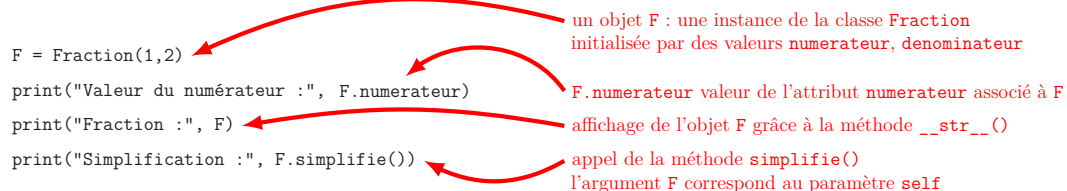
2. Application

A. Notre première classe

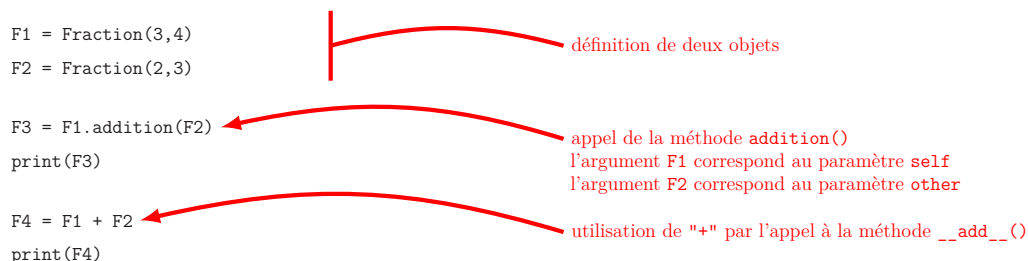


B. Nos premiers objets

Première instance



Deux autres instances



Remarque : Pour accéder aux attributs et aux méthodes d'une classe on utilise la **notation pointée**.

Cette notation pointée a déjà été vue dans votre cours de NSI : vous êtes familiers des **objets** depuis longtemps : « en Python, tout est objet » !

■ DÉFINITION

En définissant une classe on définit un **type abstrait de données**, soit une spécification mathématique :

- d'un ensemble de données
- d'un ensemble des opérations qu'on peut effectuer sur elles.

Remarque : Comme tout type abstrait, une classe possède donc une interface qui décrit l'ensemble des méthodes auxquels on a accès pour manipuler les objets de cette classe.

```
>>> L = [2, 3, 5]
>>> dir(L) # ou directement dir(list)
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__',
 '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__',
 '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__',
 '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__',
 '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__',
 '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__',
 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove',
 'reverse', 'sort']
```

On retrouve ici les méthodes applicables sur les objets du type prédéfini `list`. Vous noterez que l'interface ne précise pas la façon dont sont implémentées ces méthodes mais juste la façon de les utiliser, ce qui suffit largement généralement.

Par exemple, vous saviez déjà que pour ajouter un élément à une liste Python (qui est un objet) on pouvait utiliser la méthode `append` avec la notation pointée.

On constate aussi qu'il y a de nombreuses méthodes spéciales repérables par leur nom encadré d'un double underscore.